Package 'mathmodels'

July 29, 2026

Type Package

Title Mathematical Modeling Algorithms

Version 0.0.11

Description Implements algorithms for mathematical modeling across evaluation, prediction, and differential equation systems. It supports indicator preprocessing, weighting, TOPSIS, fuzzy comprehensive evaluation, grey relational analysis, rank sum ratio, data envelopment analysis (DEA), regional economics, inequality measures, grey prediction, Markov prediction, and time series modeling. It also provides ordinary differential equation solvers for compartmental epidemic models together with visualization and summary metrics.

License AGPL (>= 3)

URL https://github.com/zhjx19/mathmodels, https://zhjx19.github.io/mathmodels/

BugReports https://github.com/zhjx19/mathmodels/issues

Encoding UTF-8

LazyData true

Roxygen list(markdown = TRUE)

Imports lpSolveAPI,
minpack.lm,
dplyr (>= 1.0.0),
forecast,
ggplot2 (>= 3.5.0),
MASS,
purrr,
readxl,
rlang,
stats,
stringr,
tibble,
tidyr (>= 1.1.0),
tseries,
deSolve,
patchwork,
car,
lmtest

Depends R (>= 4.1)

Suggests knitr,
rmarkdown,
scales,
testthat (>= 3.0.0)

VignetteBuilder knitr

Config/testthat/edition 3

Config/roxygen2/version 8.0.0

Contents

AHP *AHP: Analytic Hierarchy Process*

Description

AHP is a multi-criteria decision analysis method developed by Saaty, which can also be used to determine indicator weights.

Usage

```
AHP(A)
```

Arguments

A	a numeric matrix, i.e. pairwise comparison matrix

Value

a list object that contains: w (Weight vector), CR (Consistency ratio), Lmax (Maximum eigenvalue), CI (Consistency index)

Examples

```
A = matrix(c(1,   1/2, 4, 3,   3,
             2,   1,   7, 5,   5,
             1/4, 1/7, 1, 1/2, 1/3,
             1/3, 1/5, 2, 1,   1,
             1/3, 1/5, 3, 1,   1), byrow = TRUE, nrow = 5)
AHP(A)
```

---

as_ts_df *Convert Common Inputs to* ts_df

Description

Converts a ts object, numeric vector, explicit data-frame columns, or an existing ts_df to the package's lightweight time-series structure.

Usage

```
as_ts_df(x, time = NULL, value = NULL, frequency = NULL, start = NULL)
```

Arguments

x	A ts, numeric vector, data frame, or ts_df.
time, value	Required column names when x is a data frame.
frequency	Optional frequency metadata.
start	Optional start metadata.

Value

A ts_df object.

Examples

```
as_ts_df(ts(1:4, frequency = 4))
as_ts_df(c(3, 5, 8))
as_ts_df(data.frame(month = 1:3, demand = c(10, 12, 15)), time = "month", value = "demand")
```

combine_preds *Combine Multiple Prediction Results*

Description

Combines multiple prediction results (e.g., from grey prediction, time series, or machine learning models) into a single prediction using a similarity-based weighting approach, improving prediction accuracy.

Usage

```
combine_preds(x)
```

Arguments

x	Numeric vector, prediction results to be combined (length >= 2).

Details

The function combines prediction results by constructing a similarity matrix based on cosine transformation of pairwise differences. Weights are derived from the principal eigenvector of the similarity matrix, ensuring predictions closer to each other have higher influence. For two predictions, equal weights (0.5, 0.5) are used. If all predictions are identical, equal weights are assigned. Compatible with the `mathmodels` package for enhancing prediction models, including grey prediction, time series, or ensemble machine learning.

Value

A list with two elements:

- a: Numeric, the combined prediction value.
- w: Numeric vector, weights for each prediction in x, summing to 1.

Examples

```
# Example: Combine three prediction results
preds = c(100, 102, 98)  # E.g., from grey prediction, ARIMA, or ML models
combine_preds(preds)
```

combine_weights *Combine Subjective and Objective Weights*

Description

Combines subjective and objective weights using linear, multiplicative, or game theory-based methods (geometric mean or linear system).

Usage

```
combine_weights(w_subj, w_obj, type = "linear", alpha = 0.5)
```

Arguments

w_subj	Numeric vector of subjective weights.
w_obj	Numeric vector of objective weights.
type	Character string specifying the combination method: "linear", "multiplicative", "game", or "game_linear".
alpha	Numeric value between 0 and 1, used only for the linear method to weight subjective weights. Defaults to 0.5.

Details

The function supports four methods:

- Linear: Combines weights as alpha * w_subj + (1 - alpha) * w_obj.
- Multiplicative: Combines weights as w_subj * w_obj, requiring positive weights.
- Game: Uses the geometric mean (sqrt(w_subj * w_obj)) to balance weights.
- Game_linear: Uses a game-theoretic approach by solving a linear system based on the cross-product of weights.

Value

A numeric vector of combined weights, normalized to sum to 1.

Examples

```
w_subj = c(0.4, 0.3, 0.2, 0.1)
w_obj = c(0.25, 0.2, 0.3, 0.25)
combine_weights(w_subj, w_obj, type = "linear", alpha = 0.6)
combine_weights(w_subj, w_obj, type = "multiplicative")
combine_weights(w_subj, w_obj, type = "game")
combine_weights(w_subj, w_obj, type = "game_linear")
```

---

complete_ts_df *Complete Missing Time Points*

Description

Adds missing time points on a regular sequence and leaves their values as NA.

Usage

```
complete_ts_df(x, by)
```

Arguments

x	A ts_df.
by	Step size passed to seq().

Value

A completed ts_df.

Examples

```
x = ts_df(data.frame(time = c(1, 2, 4), value = c(10, 12, 18)))
complete_ts_df(x, by = 1)
```

`compute_mf` *Build membership functions from knots*

Description

`compute_mf_funs` constructs a list of membership functions (one per evaluation level) from knot values. `compute_mf` evaluates a single indicator value against those functions, returning a membership vector.

Usage

```
compute_mf_funs(knots, .builder = "tri", ...)

compute_mf(x, knots, .builder = "tri", ...)
```

Arguments

`knots`	A numeric vector of length $n \geq 2$ defining the characteristic values (peaks / centers) of each evaluation level.
`.builder`	A character string or function that specifies how membership functions are built from `knots`:
	`"tri"` (default) Piecewise linear (triangular + trapezoidal). First level decays linearly from 1 to 0 across `[th[1], th[2]]`, last level rises from 0 to 1 across `[th[n-1], th[n]]`, middle levels are isosceles triangles peaking at each knot.
	`"gauss"` Gaussian (normal) membership. First level is a right-half Gaussian decaying from 1 at `th[1]`, last level is a left-half Gaussian rising to 1 at `th[n]`, middle levels are full Gaussians centered at each knot. Pass `sigma` to control spread (default = `mean(diff(knots)) / 3`).
	`"sigmoid"` Sigmoid-based membership. First level uses a decreasing sigmoid, last level an increasing sigmoid, middle levels use difference-of-sigmoids (bell-shaped). Pass `slope` to control steepness (default = `5 / mean(diff(knots))`).
	User-supplied function A function with signature `function(knots, ...)` that returns a list of n functions, each accepting a numeric vector `x` and returning membership values in $[0, 1]$.
`...`	Additional arguments passed to the builder (e.g. `sigma` for `"gauss"`, `slope` for `"sigmoid"`, or forwarded to a custom builder function).
`x`	Numeric vector, input values for which to compute membership.

Value

- `compute_mf_funs`: A list of n functions, one per level.
- `compute_mf`: A numeric vector of length n with membership degrees for each level.

Examples

```
# Triangular membership (default)
th = c(0.05, 0.15, 0.25, 0.5)
compute_mf(0.07, th)

# Gaussian membership with custom sigma
compute_mf(0.07, th, .builder = "gauss", sigma = 0.05)

# Sigmoid membership with custom slope
compute_mf(0.07, th, .builder = "sigmoid", slope = 20)

# Custom builder: exponential decay
exp_builder = function(knots, rate = 1) {
  n = length(knots)
  lapply(seq_len(n), function(i) {
    force(i)
    function(x) exp(-rate * abs(x - knots[i]))
  })
}
compute_mf(0.07, th, .builder = exp_builder, rate = 10)

## Not run:
# Visualise all levels for a given builder
mfs = compute_mf_funs(th, .builder = "gauss", sigma = 0.05)
plots = lapply(mfs, \(f) plot_mf(f, xlim = c(0, 0.6)))
gridExtra::grid.arrange(grobs = plots, nrow = 2)

## End(Not run)
```

`critic_weight` *CRITIC Weight Method*

Description

Computes objective weights of indicators and scores of samples using the CRITIC method. The method considers both the contrast intensity (e.g., standard deviation or entropy) and conflict among indicators (based on correlation) to determine indicator importance. This version supports different methods for contrast intensity and correlation types.

Usage

```
critic_weight(
  X,
  index = NULL,
  method = "std",
  cor_method = "pearson",
  epsilon = 0.002
)
```

Arguments

X	A numeric data frame or matrix where rows represent samples (observations) and columns represent indicators (variables).
index	A character vector indicating the direction of each indicator: Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better), and NA for already normalized indicators (no rescaling will be applied).
	If `index = NULL` (default), all indicators are treated as `NA`, meaning no nor
method	Character scalar; specifies the method used to compute contrast intensity. Options: "std" (standard deviation, default), or "entropy" (based on information redundancy).
cor_method	Character scalar; specifies the method for computing correlations. Options: "pearson" (default), "spearman", or "kendall".
epsilon	A small constant used to replace exact 0s and 1s in the data to prevent log(0) errors. Default is 0.002. Only used when method = "entropy".

Value

A list containing:

w	Numeric vector of weights for each indicator.
s	Numeric vector of scores for each sample (row), scaled by 100.

Examples

```
# Example: Using CRITIC method on a simple dataset
X = data.frame(
  x1 = c(3, 5, 2, 7),
  x2 = c(10, 20, 15, 25)
)
index = c("+", "-")
critic_weight(X, index)
critic_weight(X, index, method = "entropy")
```

---

curve_fit *Linearizable Curve Fit*

Description

Fits common empirical curves via variable transformation and lm(). Supported types: exponential, power-law, logarithmic, and hyperbolic.

Usage

```
curve_fit(x, y, type = c("exp", "power", "log", "hyperbolic"))
```

Arguments

x	Numeric vector of observed predictor values.
y	Numeric vector of observed response values.
type	Character, one of "exp", "power", "log", "hyperbolic".

Details

Each curve is transformed to a linear form:

- `"exp"`: fits `log(y) ~ x`, back-transforms to `y = a * exp(b * x)`
- `"power"`: fits `log(y) ~ log(x)`, back-transforms to `y = a * x^b`
- `"log"`: fits `y ~ log(x)`, gives `y = a + b * log(x)`
- `"hyperbolic"`: fits `y ~ 1/x`, gives `y = a + b / x`

Fit statistics are computed on the original scale.

Note on log-transform bias: For `"exp"` and `"power"` types, fitted values are back-transformed directly (`a * exp(b * x)` or `a * x^b`), which yields the geometric-mean prediction. As a result, predicted values on the original scale are systematically biased downward by a factor of approximately `exp(sigma^2 / 2)`. This does not affect the regression coefficients (unbiased on the log scale) or the goodness-of-fit statistics (computed on the original scale).

Note on coefficients: For `"exp"` and `"power"` types, the coefficient table reports estimates on the log-transformed scale (the parameterization of the underlying `lm()` model). The formula string shows the back-transformed parameters on the original scale.

Value

A named list, see `poly_fit()` for details.

Examples

```
set.seed(42)
x = 1:20
y = 2 * exp(0.15 * x) * rlnorm(20, sdlog = 0.1)
curve_fit(x, y, type = "exp")
```

`cv_weight` *Coefficient of Variation Weighting*

Description

Computes weights for indicators using the Coefficient of Variation (CV) method. Weights are derived by normalizing the CV (standard deviation divided by mean) for each indicator.

Usage

```
cv_weight(X)
```

Arguments

`X`	Numeric matrix or data frame with positive indicator data.

Details

The `cv_weight` function calculates weights using the CV method. For each column in `data`, the CV is computed as the standard deviation divided by the mean. Weights are obtained by normalizing the CVs to sum to 1. This lightweight implementation uses base R and assumes all columns are numeric indicators.

Value

Numeric vector of weights for the indicators, summing to 1.

Examples

```
X = data.frame(x1 = c(10, 20, 15), x2 = c(5, 10, 8))
cv_weight(X)
```

DEA *DEA efficiency analysis*

Description

DEA efficiency analysis

Usage

```
basic_DEA(
  data,
  inputs,
  outputs,
  ud_outputs = NULL,
  orientation = "io",
  rts = "vrs"
)

super_DEA(data, inputs, outputs, orientation = "io", rts = "vrs")

basic_SBM(
  data,
  inputs,
  outputs,
  ud_outputs = NULL,
  orientation = "io",
  rts = "vrs"
)

super_SBM(data, inputs, outputs, orientation = "io", rts = "vrs")

malmquist(
  data,
  period,
  inputs,
  outputs,
  orientation = "oo",
  rts = "vrs",
  type1 = "glob",
  type2 = "rd"
)
```

Arguments

data	A data frame. 1st column = DMU names.
inputs	Column indices/names of input variables.
outputs	Column indices/names of output variables.
ud_outputs	Column indices (within outputs) of undesirable outputs, or NULL.
orientation	"io" (input-oriented, default) or "oo" (output-oriented). All DEA functions return Farrell efficiency (0~1), where 1 = efficient. For output orientation: value = $1/\phi$, where $\phi \geq 1$ is the Shephard distance.
rts	"vrs" (variable returns to scale, default) or "crs" (constant).
period	Column name or index identifying the time period variable.
type1	"cont" (contemporaneous), "seq" (sequential), or "glob" (global, default).
type2	"fgnz" (Färe-Grosskopf-Norris-Zhang) or "rd" (Ray-Desli, default).

Value

A list with components: efficiencies, lambdas, slacks, targets, returns, model, orientation, dmu.

Functions

- basic_DEA(): Standard radial DEA (CCR/BCC)
- super_DEA(): Super-efficiency DEA (self excluded, radial only)
- basic_SBM(): Standard Slacks-Based Measure (SBM, Tone 2001)
- super_SBM(): Super-efficiency SBM (self excluded, no undesirable outputs)
- malmquist(): Malmquist productivity index

Examples

```
df = data.frame(
  DMU = paste0("DMU", 1:7),
  x1  = c(20, 60, 40, 60, 70, 30, 50),
  x2  = c(151, 200, 120, 170, 250, 210, 90),
  y1  = c(100, 210, 150, 240, 220, 80, 200)
)
basic_DEA(df, inputs = 2:3, outputs = 4, rts = "crs")
basic_DEA(df, inputs = 2:3, outputs = 4, rts = "vrs")
df = data.frame(
  DMU = paste0("DMU", 1:7),
  x1  = c(20, 60, 40, 60, 70, 30, 50),
  x2  = c(151, 200, 120, 170, 250, 210, 90),
  y1  = c(100, 210, 150, 240, 220, 80, 200)
)
super_DEA(df, inputs = 2:3, outputs = 4, rts = "crs")
df = data.frame(
  DMU = paste0("DMU", 1:7),
  x1  = c(20, 60, 40, 60, 70, 30, 50),
  x2  = c(151, 200, 120, 170, 250, 210, 90),
  y1  = c(100, 210, 150, 240, 220, 80, 200)
)
basic_SBM(df, inputs = 2:3, outputs = 4, rts = "crs")
df = data.frame(
```

```
  DMU = paste0("DMU", 1:7),
  x1  = c(20, 60, 40, 60, 70, 30, 50),
  x2  = c(151, 200, 120, 170, 250, 210, 90),
  y1  = c(100, 210, 150, 240, 220, 80, 200)
)
super_SBM(df, inputs = 2:3, outputs = 4, rts = "crs")
panel = data.frame(
  DMU    = rep(paste0("DMU", 1:5), 3),
  Period = rep(1:3, each = 5),
  x1     = c(10, 20, 15, 25, 30, 12, 22, 17, 27, 32, 14, 24, 19, 29, 34),
  y1     = c(100, 150, 120, 180, 200, 110, 160, 130, 190, 210, 120, 170, 140, 200, 220)
)
malmquist(panel, period = "Period", inputs = 3, outputs = 4,
  rts = "crs", type1 = "cont", type2 = "fgnz")
```

`defuzzify` *Defuzzification Methods for Fuzzy Comprehensive Evaluation*

Description

Implements defuzzification methods for fuzzy evaluation vectors, including weighted average and maximum membership methods.

Usage

```
defuzzify(mu, scores, method = "weighted_average")
```

Arguments

`mu`	Numeric vector, membership degrees for evaluation levels, in `[0, 1]`.
`scores`	Numeric vector, scores corresponding to each evaluation level (e.g., c(100, 80, 60, 40) for "Excellent", "Good", "Fair", "Poor").
`method`	Character, defuzzification method: "weighted_average", "max_membership", "centroid".

Value

Numeric, defuzzified output value.

Examples

```
# Example: Defuzzify fuzzy evaluation vectors for three schemes
mu = c(0.318, 0.351, 0.203, 0.128)
scores = c(30, 60, 75, 90)  # Scores for "Poor", "Fair", "Good", "Excellent"
defuzzify(mu, scores, method = "weighted_average")
defuzzify(mu, scores, method = "max_membership")
defuzzify(mu, scores, method = "centroid")
```

drop_na_ts_df *Drop Missing Values from a* ts_df

Description

Drop Missing Values from a ts_df

Usage

```
drop_na_ts_df(x)
```

Arguments

x	A ts_df.

Value

A ts_df with rows whose value is NA removed.

Examples

```
x = ts_df(data.frame(time = 1:4, value = c(10, NA, 12, 13)))
drop_na_ts_df(x)
```

entropy_weight *Entropy Weight Method*

Description

Computes the weights of indicators and scores of samples based on the entropy method. This method objectively determines the importance of each indicator according to the amount of information it contains.

Usage

```
entropy_weight(X, index = NULL, epsilon = 0.002)
```

Arguments

X	A numeric data frame or matrix where rows represent samples (observations) and columns represent indicators (variables).
index	A character vector indicating the direction of each indicator. Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better), and NA for already normalized indicators (no rescaling will be applied, but minor adjustments will still be made to avoid log(0) errors). If index = NULL (default), all indicators are treated as NA, meaning no normalization or rescaling is performed, but a small adjustment is still applied to prevent log(0) errors.
epsilon	A small constant used to replace exact 0s and 1s in the data to prevent log(0) errors. Default is 0.002.

Value

A list containing:

w	Numeric vector of weights for each indicator.
s	Numeric vector of scores for each sample (row), scaled by 100.

Examples

```
X = data.frame(
  x1 = c(3, 5, 2, 7),
  x2 = c(10, 20, 15, 25)
)
index = c("+", "-")
entropy_weight(X, index)
```

---

epi_metrics *Extract key epidemic metrics from ODE simulation output*

Description

This function computes core epidemiological summary statistics from deterministic compartmental epidemic models (e.g., SIR, SIS, SEIR).

Usage

```
epi_metrics(data, beta, gamma, N = NULL)
```

Arguments

data	A data frame returned by `model_*()` functions. Must contain a column `time`, and at least `S` and `I`. Additional compartments such as `E` or `R` are allowed.
beta	Transmission rate (numeric).
gamma	Recovery / removal rate (numeric).
N	Total population (numeric). If `NULL` (default), it is computed as the sum of all compartment values at the first time point.

Details

The input data frame is expected to be the output of `model_*()` functions, containing a time column and one or more compartment columns (e.g., S, I, R, E). All compartment values are assumed to be in absolute population counts, and the total population is assumed to be conserved unless otherwise specified.

Value

A named list with:

R0 Basic reproduction number.

peak_infection Maximum number of infectious individuals.

peak_time Time at which infectious peak occurs.

attack_rate Proportion of population that transitioned out of S; defined only for models containing an R compartment (e.g., SIR/SEIR).

Examples

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
epi_metrics(sir, beta = 0.002, gamma = 0.1)
```

---

fuzzy_eval *Fuzzy Comprehensive Evaluation*

Description

Performs fuzzy comprehensive evaluation using different fuzzy composition operators to combine factor weights with a fuzzy evaluation matrix. Suitable for multi-criteria decision analysis with weights from methods like AHP, entropy, CRITIC, CV, or PCA.

Usage

```
fuzzy_eval(w, R, type)
```

Arguments

w	Numeric vector, factor weights (e.g., from combine_weights).
R	Numeric matrix, fuzzy evaluation matrix with columns as factors and rows as evaluation grades. Values should be in [0, 1].
type	Integer or character (1-5), specifying the fuzzy composition operator: • 1: Min-max (main factor decisive). • 2: Product-max (main factor prominent). • 3: Weighted sum (additive average). • 4: Bounded sum of mins (min-sum bounded). • 5: Normalized min-sum (balanced average).

Details

The function computes a fuzzy comprehensive evaluation vector B based on the weight vector w and fuzzy evaluation matrix R. Five composition operators are supported:

- Type 1 (min-max): max(min(w, R[j,])), emphasizes the main factor.
- Type 2 (product-max): max(w * R[j,]), highlights the main factor.
- Type 3 (weighted sum): sum(w * R[j,]), additive average.
- Type 4 (bounded sum): min(1, sum(min(w, R[j,]))), bounds the sum of mins.
- Type 5 (normalized min-sum): sum(min(w, R[j,]/sum(R[j,]))), balanced average.

The output B is normalized to sum to 1. If the sum is zero, an error is thrown. Uses base R for lightweight implementation.

Value

A numeric vector of normalized comprehensive evaluation results, summing to 1.

Examples

```
w = c(0.3, 0.3, 0.3, 0.1)  # weights (e.g., from AHP or entropy)

# fuzzy evaluation matrix (3 grades for 4 factors)
R = matrix(c(0.8, 0.7, 0.6, 0.7,
             0.1, 0.2, 0.2, 0.1,
             0.1, 0.1, 0.2, 0.2), nrow = 3, byrow = TRUE)
# Apply fuzzy comprehensive evaluation
fuzzy_eval(w, R, type = 3)  # Weighted sum
```

GM11_markov *Grey-Markov Prediction Model*

Description

Fits a GM(1,1) model and then uses a Markov chain on relative error states to correct both historical fitted values and future predictions.

Usage

```
GM11_markov(X, n_ahead = 3, breaks = c(-0.1, -0.05, -0.02, 0, 0.02, 0.05, 0.1))
```

Arguments

X	Numeric vector of original time series data.
n_ahead	Number of future periods to predict. Must be a positive integer.
breaks	Numeric vector of boundaries for classifying relative errors into states. -Inf and Inf are prepended/appended internally.

Details

The function first fits a GM(1,1) model to the original series, classifies relative errors into states using breaks, builds a Markov chain on these error states, and finally adjusts the GM(1,1) fitted and predicted values.

Value

A data frame with columns:

Period Time period labels, such as T1, T2, ..., T+n.

Raw Original values; future periods are NA.

GM11_fitted GM(1,1) fitted or predicted values.

err_state Relative error state for historical periods, and predicted state for future periods.

adj_eff Adjustment effect, defined as the midpoint of the state interval.

Markov_adj Markov-chain-adjusted fitted or predicted values.

Examples

```
X = c(174, 179, 183, 189, 207, 234, 220.5, 256, 270, 285)
GM11_markov(X, n_ahead = 3)
```

`grey_analysis` *Grey Relational Analysis Functions*

Description

A collection of functions for performing grey relational analysis, including calculation of grey correlation degree and evaluation based on grey correlation. These functions are designed for decision-making and data analysis by measuring the relational degree between sequences.

Usage

```
grey_corr(ref, cmp, rho = 0.5, w = NULL)

grey_corr_topsis(X, w, index = NULL, rho = 0.5)
```

Arguments

`ref`	Numeric vector, the reference sequence for `grey_corr`.
`cmp`	Numeric matrix or data frame, the comparison sequences for `grey_corr`.
`rho`	Numeric scalar, the distinguishing coefficient (default = 0.5).
`w`	Numeric vector, weights for weighted correlation (default = equal weights).
`X`	Numeric matrix or data frame, the decision matrix for `grey_corr_topsis`.
`index`	Character vector indicating indicator direction: Use `"+"` for positive indicators (higher is better), `"-"` for negative indicators (lower is better), and `NA` for already rescaled indicators (no rescaling will be applied). If `index = NULL` (default), all indicators are treated as `NA`, meaning no rescaling is performed.

Details

These functions implement grey relational analysis for evaluating relationships between sequences or decision alternatives:

grey_corr Computes the grey correlation degree between a reference sequence (`ref`) and comparison sequences (`cmp`) using the distinguishing coefficient (`rho`) and optional weights (`w`).

grey_corr_topsis Evaluates a decision matrix (`X`) by normalizing it, applying weights (`w`), computing grey correlation with the ideal sequence. Direction of indicators can be specified via `index`.

Value

grey_corr Returns a numeric vector of grey correlation degrees for each comparison sequence.

grey_corr_topsis Returns a numeric vector of relative closeness (grey correlation degrees).

Examples

```
# Grey correlation degree
ref = c(0.9, 0.8, 0.7)
cmp = data.frame(
  x1 = c(0.9, 0.7, 0.8),
  x2 = c(0.8, 0.9, 0.7),
  x3 = c(0.7, 0.8, 0.9)
)
grey_corr(ref, cmp, rho = 0.5)

# Grey correlation evaluation
X = data.frame(x1 = c(8, 7, 6), x2 = c(150, 180, 200), x3 = c(60, 80, 100))
w = c(0.3, 0.4, 0.3)
idx = c("+", "+", "+")
grey_corr_topsis(X, w, idx, rho = 0.5)
```

---

grey_models *Grey Prediction Models*

Description

Implements grey prediction models for time series forecasting: `GM11` applies the GM(1,1) model with level ratio test. `GM1N` applies the GM(1,N) model with multiple related factors. `DGM21` applies the DGM(2,1) model for second-order dynamics. `verhulst` applies the Verhulst model for logistic growth.

Usage

```
GM11(X)

GM1N(dat, new_data = NULL)

DGM21(X)

verhulst(X)
```

Arguments

`X`	For `GM11`, `DGM21`, `verhulst`: Numeric vector of original time series data.
`dat`	For `GM1N`: Data frame or matrix, last column is characteristic series, others are related factors.
`new_data`	For `GM1N`: Optional future values of related factors (1 row, m-1 columns).

Value

For `GM11`: List with fitted values (`fitted`), next prediction (`pnext`), prediction function (`f`), matrix (`mat`), parameters (`u`), level ratios (`lambda`), and range (`rng`). For `GM1N`: List with fitted values (`fitted`), posterior variance ratio (`C`), small error probability (`P`), and prediction function (`f`). For `DGM21`, `verhulst`: List with fitted values (`fitted`), next prediction (`pnext`), prediction function (`f`), matrix (`mat`), and parameters (`u`).

Examples

```
# Sample time series for GM11, DGM21, Verhulst
x = c(100, 120, 145, 175, 210)

# GM11
result = GM11(x)
result$fitted    # Fitted values
result$pnext     # Next prediction
result$f(6:8)    # Predict next 3 periods

# DGM21
x = c(2.874,3.278,3.39,3.679,3.77,3.8)
result = DGM21(x)
result$fitted    # Fitted values
result$pnext     # Next prediction
result$f(6:8)    # Predict next 3 periods

# Verhulst
x = c(4.93,2.33,3.87,4.35,6.63,7.15,5.37,6.39,7.81,8.35)
result = verhulst(x)
result$fitted    # Fitted values
result$pnext     # Next prediction
result$f(6:8)    # Predict next 3 periods

# Sample data for GM1N
data = data.frame(
  factor1 = c(50, 55, 60, 65, 70),
  factor2 = c(20, 22, 25, 28, 30),
  output = c(100, 120, 145, 175, 210)
)
result = GM1N(data)
result$fitted
```

---

growth_fit *Nonlinear Growth Curve Fit*

Description

Fits nonlinear growth, saturation, and sigmoid curves via `minpack.lm::nlsLM()` with automatic starting value generation.

Usage

```
growth_fit(x, y, type = c("logistic", "gompertz", "saturation", "mm"))
```

Arguments

`x`	Numeric vector of observed predictor values.
`y`	Numeric vector of observed response values.
`type`	Character, one of `"logistic"`, `"gompertz"`, `"saturation"`, or `"mm"`.

Details

Models and starting value strategies:

Logistic: `y = L / (1 + exp(-k * (x - x0)))`

- `L0 = max(y) * 1.05`
- `lm(log((L0 - y) / y) ~ x)` for `k, x0` initials

Gompertz: `y = L * exp(-exp(-k * (x - x0)))`

- `L0 = max(y) * 1.05`
- `lm(log(-log(y / L0)) ~ x)` for `k, x0` initials

Saturation (exponential approach): `y = a * (1 - exp(-b * x))`

- `a0 = max(y) * 1.1`
- `lm(log(a0 - y) ~ x)` for `b` initial

Michaelis-Menten: `y = a * x / (b + x)`

- `a0 = max(y) * 1.1`
- Lineweaver-Burk `lm(1/y ~ 1/x)` for `a, b` initials

Falls back to heuristic defaults if linearized estimation fails.

Value

A named list, see `poly_fit()` for details. `model` contains the `nls` object.

Examples

```
set.seed(42)
x = seq(0, 10, length.out = 30)
y = 100 / (1 + exp(-1.5 * (x - 5))) + rnorm(30, sd = 2)
growth_fit(x, y, type = "logistic")
```

`impute_ts_df` *Complete and Interpolate a* `ts_df`

Description

Completes missing time points and fills internal missing values by linear or spline interpolation. Endpoint values must already be observed.

Usage

```
impute_ts_df(x, by, method = c("linear", "spline"))
```

Arguments

`x`	A `ts_df`.
`by`	Step size passed to `seq()`.
`method`	Interpolation method: `"linear"` or `"spline"`.

Value

An interpolated `ts_df`.

Examples

```
x = ts_df(data.frame(time = c(1, 3, 5), value = c(10, 20, 30)))
impute_ts_df(x, by = 1, method = "linear")
```

`inequality` *Inequality Indices*

Description

Computes inequality indices for individual or grouped data: `gini0` calculates the Gini coefficient for individual sample data. `gini` calculates the Gini coefficient for grouped data using income and population shares. `theil0` calculates the Theil index for individual sample data. `theil` calculates the Theil index for grouped average data. `theil0_g` calculates the Theil index and decomposition for grouped sample data. `theil_g` calculates the Theil index and decomposition for grouped average data. `theil_g2_cross` calculates the Theil index and decomposition for two-level cross-grouped average data. `theil_g2_nest` calculates the Theil index and decomposition for two-level nested grouped average data.

For `theil`, `theil_g`, `theil_g2_cross`, `theil_g2_nest`: Name of population variable (character).

Usage

```
gini0(y)

gini(y, pop)

theil0(y)

theil(y, pop)

theil0_g(data, group, y)

theil_g(data, group, y, pop)

theil_g2_cross(data, group1, group2, y, pop)

theil_g2_nest(data, group1, group2, y, pop)
```

Arguments

`y`	For `gini0`, `gini`, `theil0`: Numeric vector of individual incomes.
`pop`	For `gini`: Numeric vector of group populations or population shares. For `theil`, `theil0_g`, `theil_g`, `theil_g2_cross`, `theil_g2_nest`: Name of income variable (character).
`data`	For `theil0_g`, `theil_g`, `theil_g2_cross`, `theil_g2_nest`: Data frame containing variables.
`group`	For `theil0_g`, `theil_g`: Name of grouping variable (e.g., province).

group1 For theil_g2_cross, theil_g2_nest: Name of first grouping variable (e.g., region or province).

group2 For theil_g2_cross, theil_g2_nest: Name of second grouping variable (e.g., type or city).

Value

For gini0, gini: Numeric Gini coefficient (0 to 1). For theil0, theil: Numeric Theil index. For theil0_g, theil_g, theil_g2_cross, theil_g2_nest: List with two vectors:

- theil: theil (Theil index and its decomposition),
- ratio: ratio (contribution rates of each component).

Examples

```
# Sample data
income = c(10, 20, 30, 40, 100)
pop = c(100, 150, 200, 250, 300)

# Gini coefficient (individual data)
gini0(income)

# Gini coefficient (grouped data)
gini(income, pop)

# Theil index (individual sample)
data = data.frame(g = c("A","A",rep("B",10),rep("A",6)),
                  y = c(10,10,rep(8,4),rep(6,6),rep(4,4),2,2))
theil0(data$y)

# Theil index (grouped average)
data2 = data |> dplyr::count(g, y, name = "pop")
theil(data2$y, data2$pop)

# Theil index with grouping (sample data)
theil0_g(data, "g", "y")

# Theil index with grouping (average data)
theil_g(data2, "g", "y", "pop")

# Theil index with two-level cross-grouping
data3 = data.frame(
  industry = c("A", "A", "A", "A", "B", "B", "B", "B"),
  area = c("East", "East", "West", "West", "East", "East", "West", "West"),
  province = c("Shanghai", "Beijing", "Sichuan", "Yunnan",
               "Shanghai", "Beijing", "Sichuan", "Yunnan"),
  avg_wage = c(500, 400, 80, 60, 300, 250, 50, 40),
  emp_num = c(100, 80, 90, 70, 120, 100, 80, 60)
)
theil_g2_cross(data3, "industry", "area", "avg_wage", "emp_num")

# Theil index with two-level nested grouping
data4 = data.frame(
  province = c("A", "A", "A", "A", "B", "B"),
  city = c("A1", "A1", "A2", "A2", "B1", "B1"),
  industry = c("Manu", "Serv", "Manu", "Serv", "Manu", "Serv"),
```

```
  y = c(50000, 45000, 60000, 55000, 70000, 65000),
  pop = c(10000, 8000, 15000, 12000, 10000, 8000)
)
theil_g2_nest(data4, "province", "city", "y", "pop")
```

---

interp_hermite *Piecewise Cubic Hermite Interpolation*

Description

Shape-preserving (monotone) piecewise cubic Hermite interpolation using the Fritsch-Carlson method via `splinefun(method = "monoH.FC")`.

Usage

```
interp_hermite(x, y, xout)
```

Arguments

`x`	Numeric vector of observed x-values (no duplicates, length >= 2).
`y`	Numeric vector of observed y-values.
`xout`	Numeric vector of x-values where interpolation is desired.

Value

A tibble with columns `x` and `y`. The attribute `method` is set to `"hermite"`.

Examples

```
x = c(1, 2, 3, 4, 5)
y = c(1, 2, 4, 7, 11)
interp_hermite(x, y, xout = c(1.5, 2.5, 3.5))
```

---

interp_linear *Linear Interpolation*

Description

Performs linear (piecewise) interpolation at specified points.

Usage

```
interp_linear(x, y, xout)
```

Arguments

`x`	Numeric vector of observed x-values (no duplicates).
`y`	Numeric vector of observed y-values, same length as `x`.
`xout`	Numeric vector of x-values where interpolation is desired.

Value

A tibble with columns `x` and `y`, containing the interpolated values at `xout`. The attribute `method` is set to `"linear"`.

Examples

```
x = c(1, 2, 3, 4, 5)
y = c(2, 4, 1, 5, 3)
interp_linear(x, y, xout = c(1.5, 2.5, 3.5))
```

`interp_poly` *Polynomial Interpolation*

Description

Fits a polynomial of specified degree via `lm()` and evaluates it at `xout`.

Usage

```
interp_poly(x, y, xout, degree = 3)
```

Arguments

`x`	Numeric vector of observed x-values (no duplicates).
`y`	Numeric vector of observed y-values.
`xout`	Numeric vector of x-values where interpolation is desired.
`degree`	Integer, degree of the polynomial. Minimum 1.

Value

A tibble with columns `x` and `y`. The attribute `method` is set to `"poly"`.

Examples

```
x = c(1, 2, 3, 4, 5)
y = c(2, 4, 1, 5, 3)
interp_poly(x, y, xout = c(1.5, 2.5, 3.5), degree = 3)
```

interp_spline *Cubic Spline Interpolation*

Description

Performs cubic spline interpolation. When spar is NULL, uses stats::spline() with the FMM method. Otherwise uses stats::smooth.spline() for smoothing.

Usage

```
interp_spline(x, y, xout, spar = NULL)
```

Arguments

x	Numeric vector of observed x-values (no duplicates, length >= 4).
y	Numeric vector of observed y-values.
xout	Numeric vector of x-values where interpolation is desired.
spar	Smoothing parameter passed to smooth.spline(). NULL for standard (non-smoothing) spline.

Value

A tibble with columns x and y. The attribute method is set to "spline" or "smooth_spline" depending on the variant used.

Examples

```
x = c(1, 2, 3, 4, 5)
y = c(2, 4, 1, 5, 3)
interp_spline(x, y, xout = c(1.5, 2.5, 3.5))
interp_spline(x, y, xout = c(1.5, 2.5, 3.5), spar = 0.8)
```

is_ts_df *Check Whether an Object Is a* ts_df

Description

Check Whether an Object Is a ts_df

Usage

```
is_ts_df(x)
```

Arguments

x	Object to check.

Value

TRUE or FALSE.

Examples

```
x = ts_df(data.frame(time = 1:3, value = c(10, 12, 15)))
is_ts_df(x)
```

---

linear_sum *Linear weighted synthesis*

Description

Linear weighted synthesis

Usage

```
linear_sum(data, w)
```

Arguments

data	Indicator matrix or data frame (rows = samples, columns = indicators)
w	Weight vector (length must match ncol(data))

Value

Vector of comprehensive scores

Examples

```
data = data.frame(
  GDP = c(0.85, 0.72, 0.91, NA),
  Employment = c(0.78, 0.85, 0.67, 0.73),
  Environment = c(0.65, 0.72, NA, 0.81)
)
w = c(0.5, 0.3, 0.2)
linear_sum(data, w)
```

---

markov_chain *Markov Chain Prediction*

Description

Constructs a transition probability matrix from a state sequence and performs multi-step Markov chain prediction.

Usage

```
markov_chain(S, s0, n_steps = 5)
```

Arguments

S	State sequence, supplied as a vector or factor.
s0	Initial state, must be one of the state levels in S.
n_steps	Number of prediction steps. Must be a positive integer.

Details

For states that never appear as a starting state, equal transition probabilities are assigned to keep each row sum equal to 1. The ultimate stationary distribution is computed using eigenvalue decomposition.

Value

A list with components:

trans_mat Transition probability matrix.

pred_probs Matrix of state probabilities for each prediction step.

pred_states Predicted states, taking the most likely state at each step.

pi_final Ultimate stationary distribution.

Examples

```
# Weather states: Rainy, Cloudy, Sunny
S = factor(c("Sunny", "Sunny", "Cloudy", "Rainy", "Sunny",
             "Cloudy", "Sunny", "Sunny", "Rainy", "Cloudy",
             "Sunny", "Cloudy", "Rainy", "Sunny", "Sunny",
             "Cloudy", "Sunny", "Rainy", "Cloudy", "Sunny"),
           levels = c("Rainy", "Cloudy", "Sunny"))
markov_chain(S, s0 = "Cloudy", n_steps = 3)
```

---

membership *Membership Functions for Fuzzy Logic*

Description

A collection of functions to compute membership values for various fuzzy sets, including triangular, trapezoidal, Gaussian, generalized bell, two-parameter Gaussian, sigmoid, difference of sigmoids, product of sigmoids, Z-shaped, PI-shaped, and S-shaped membership functions. Includes a function to visualize membership functions using ggplot2. These are designed for evaluation models in mathematical modeling, compatible with `fuzzy_eval` in the `mathmodels` package.

Usage

```
tri_mf(x, params)

trap_mf(x, params)

gauss_mf(x, params)
```

```
gbell_mf(x, params)

gauss2mf(x, params)

sigmoid_mf(x, params)

dsigmoid_mf(x, params)

psigmoid_mf(x, params)

z_mf(x, params)

pi_mf(x, params)

s_mf(x, params)

plot_mf(mf, xlim = c(0, 10), main = NULL)
```

Arguments

x	Numeric vector, input values for which to compute membership.
params	Numeric vector, parameters defining the membership function: • For tri_mf: c(a, b, c), where a <= b <= c (left base, peak, right base). • For trap_mf: c(a, b, c, d), where a <= b <= c <= d (left base, left top, right top, right base). • For gauss_mf: c(sigma, c), where sigma > 0 (spread, center). • For gbell_mf: c(a, b, c), where a > 0, b > 0 (width, shape, center). • For gauss2mf: c(s1, c1, s2, c2), where s1 > 0, s2 > 0 (left spread, left center, right spread, right center). • For sigmoid_mf: c(a, b), where a > 0 (slope, inflection point). • For dsigmoid_mf: c(a1, c1, a2, c2), where a1 > 0, a2 > 0 (slopes and inflection points for two sigmoids). • For psigmoid_mf: c(a1, c1, a2, c2), where a1 > 0, a2 > 0 (slopes and inflection points for two sigmoids). • For z_mf: c(a, b), where a < b (left base, right base). • For pi_mf: c(a, b, c, d), where a < b < c < d (left base, left shoulder, right shoulder, right base). • For s_mf: c(a, b), where a < b (left base, right base).
mf	Function, a membership function with fixed parameters (e.g., function(x) tri_mf(x, c(2, 5, 8))).
xlim	Numeric vector of length 2, x-axis limits for plotting (default c(0, 10)).
main	Character, plot title (default NULL, no title).

Details

These functions support evaluation models in mathematical modeling:

- tri_mf: Triangular membership, linear rise from a to b (peak) and fall to c.
- trap_mf: Trapezoidal membership, linear rise from a to b, plateau from b to c, fall to d.
- gauss_mf: Gaussian membership, bell-shaped curve centered at c with spread sigma.

- gbell_mf: Generalized bell membership, bell-shaped curve with width a, shape b, and center c.
- gauss2mf: Two-parameter Gaussian membership, combining two Gaussians with spreads s1, s2 and centers c1, c2.
- sigmoid_mf: Sigmoid membership, S-shaped curve with slope a and inflection point b.
- dsigmoid_mf: Difference of two sigmoids, combining slopes a1, a2 and inflection points c1, c2.
- psigmoid_mf: Product of two sigmoids, combining slopes a1, a2 and inflection points c1, c2.
- z_mf: Z-shaped membership, decreasing from 1 at a to 0 at b.
- pi_mf: PI-shaped membership, rising from a to b, plateau from b to c, falling to d.
- s_mf: S-shaped membership, increasing from 0 at a to 1 at b.
- plot_mf: Plots a membership function over xlim using ggplot2, suitable for tidyverse workflows.

Membership values can be used to construct fuzzy evaluation matrices for fuzzy_eval. Implemented in base R, except plot_mf, which requires ggplot2.

Value

- For membership functions (tri_mf, trap_mf, gauss_mf, gbell_mf, gauss2mf, sigmoid_mf, dsigmoid_mf, psigmoid_mf, z_mf, pi_mf, s_mf): A numeric vector of membership values in [0, 1], same length as x.
- For plot_mf: A ggplot2 object, plotting the membership function.

Examples

```
# Define input values
x = 0:10

# Triangular membership
tri_mf(x, params = c(3, 6, 8))

# Trapezoidal membership
trap_mf(x, params = c(1, 5, 7, 8))

# Gaussian membership
gauss_mf(x, params = c(2, 5))

# Generalized bell membership
gbell_mf(x, params = c(2, 4, 6))

# Two-parameter Gaussian membership
gauss2mf(x, params = c(1, 3, 3, 4))

# Sigmoid membership
sigmoid_mf(x, params = c(2, 4))

# Difference of sigmoids membership
dsigmoid_y = dsigmoid_mf(x, params = c(5, 2, 5, 7))

# Product of sigmoids membership
psigmoid_mf(x, params = c(2, 3, -5, 8))
```

```
# Z-shaped membership
z_mf(x, params = c(3, 7))

# PI-shaped membership
pi_mf(x, params = c(1, 4, 5, 10))

# S-shaped membership
s_mf(x, params = c(1, 8))

## Not run:
# Visualize membership functions
plot_mf(\(x) tri_mf(x, c(3, 6, 8)), main = "Triangular MF")
plot_mf(\(x) trap_mf(x, c(1, 5, 7, 8)), main = "Trapezoidal MF")
plot_mf(\(x) gauss_mf(x, c(2, 5)), main = "Gaussian MF")
plot_mf(\(x) gbell_mf(x, c(2, 4, 6)), main = "Generalized Bell MF")
plot_mf(\(x) gauss2mf(x, c(1, 3, 3, 4)), main = "Two-Parameter Gaussian MF")
plot_mf(\(x) sigmoid_mf(x, c(2, 4)), main = "Sigmoid MF")
plot_mf(\(x) dsigmoid_mf(x, c(5, 2, 5, 7)), main = "Difference of Sigmoids MF")
plot_mf(\(x) psigmoid_mf(x, c(2, 3, -5, 8)), main = "Product of Sigmoids MF")
plot_mf(\(x) z_mf(x, c(3, 7)), main = "Z-Shaped MF")
plot_mf(\(x) pi_mf(x, c(1, 4, 5, 10)), main = "PI-Shaped MF")
plot_mf(\(x) s_mf(x, c(1, 8)), main = "S-Shaped MF")

## End(Not run)
```

`model_logistic` *Logistic Population Growth Model*

Description

Solves the logistic growth equation:

$$\frac{dN}{dt} = r\left(1 - \frac{N}{K}\right)N$$

Usage

```
model_logistic(init, params, times, ...)
```

Arguments

`init`	Named numeric vector, e.g. `c(N = 10)`.
`params`	Named numeric vector, e.g. `c(r = 0.5, K = 100)`.
	r Intrinsic growth rate.
	K Carrying capacity.
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

The analytical solution is $N(t) = K \,/\, \left(1+(K/N_0-1)\,e^{-rt}\right)$, which can be used to verify numerical accuracy.

Value

A `data.frame` with columns `time` and `N`.

Examples

```
res = model_logistic(
  init   = c(N = 10),
  params = c(r = 0.5, K = 100),
  times  = seq(0, 20, by = 0.1)
)
head(res)
```

`model_lv` *Lotka-Volterra Predator-Prey Model*

Description

Classic two-species predator-prey system:

$$\frac{dx}{dt} = \alpha\, x - \beta\, xy$$

$$\frac{dy}{dt} = \delta\, xy - \mu\, y$$

Usage

```
model_lv(init, params, times, ...)
```

Arguments

`init`	Named numeric vector, e.g. `c(x = 40, y = 9)`.
`params`	Named numeric vector with four parameters: **alpha** Prey intrinsic growth rate. **beta** Predation rate (prey killed per predator per unit time). **delta** Predator reproduction rate per prey consumed. **mu** Predator mortality rate. Named `mu` (not `gamma`) to avoid confusion with the recovery rate used in epidemic models.
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

- **x** Prey population.
- **y** Predator population.

Value

A `data.frame` with columns `time`, `x`, `y`.

Examples

```
model_lv(
  init   = c(x = 40, y = 9),
  params = c(alpha = 0.4, beta = 0.04, delta = 0.02, mu = 0.5),
  times  = seq(0, 100, by = 0.1)
)
```

`model_malthus` *Malthusian (Exponential) Growth Model*

Description

Solves the Malthusian growth equation:

$$\frac{dN}{dt} = r\,N$$

Usage

```
model_malthus(init, params, times, ...)
```

Arguments

`init`	Named numeric vector, e.g. `c(N = 100)`.
`params`	Named numeric vector, e.g. `c(r = 0.3)`.
	r Intrinsic growth rate (per time unit).
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

The analytical solution is $N(t) = N_0\,e^{rt}$, which can be used to verify numerical accuracy.

Value

A `data.frame` with columns `time` and `N`.

Examples

```
res = model_malthus(
  init   = c(N = 100),
  params = c(r = 0.3),
  times  = seq(0, 10, by = 0.1)
)
# Compare with analytical solution
res$N_exact = 100 * exp(0.3 * res$time)
head(res)
```

`model_seir` *SEIR Epidemic Model*

Description

Four-compartment model that adds an exposed (latent) class.

Usage

```
model_seir(init, params, times, ...)
```

Arguments

`init`	Named numeric vector. `R` defaults to 0 if omitted, e.g. `c(S = 980, E = 10, I = 10)`.
`params`	Named numeric vector, e.g. `c(beta = 0.3, alpha = 0.2, gamma = 0.1)`. **beta** Transmission rate. **alpha** Rate of progression from exposed to infectious (1/alpha = mean latent period). **gamma** Recovery rate (1/gamma = mean infectious period). **N** Total population size. Optional — defaults to `sum(init)`. **b** Birth rate (per capita per time). Optional. **d** Death rate (per capita per time). Optional.
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

Closed population (default):

$$\frac{dS}{dt} = -\beta \frac{SI}{N}$$

$$\frac{dE}{dt} = \beta \frac{SI}{N} - \alpha E$$

$$\frac{dI}{dt} = \alpha E - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

Open population (when b / d are provided):

$$\frac{dS}{dt} = bN - \beta \frac{SI}{N} - dS$$

$$\frac{dE}{dt} = \beta \frac{SI}{N} - \alpha E - dE$$

$$\frac{dI}{dt} = \alpha E - \gamma I - dI$$

$$\frac{dR}{dt} = \gamma I - dR$$

- **S** Susceptible — not yet infected.
- **E** Exposed — infected but not yet infectious (mean latent period $1/\alpha$).
- **I** Infectious — actively spreading the pathogen (mean infectious period $1/\gamma$).
- **R** Recovered — permanently immune.

In the closed case population size $N = S + E + I + R$ is conserved.

Value

A `data.frame` with columns `time`, `S`, `E`, `I`, `R`.

Examples

```
# Closed population
model_seir(
  init   = c(S = 980, E = 10, I = 10),
  params = c(beta = 0.3, alpha = 0.2, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)

# Open population with births and deaths
model_seir(
  init   = c(S = 980, E = 10, I = 10),
  params = c(beta = 0.3, alpha = 0.2, gamma = 0.1, b = 0.01, d = 0.01),
  times  = seq(0, 200, by = 1)
)
```

`model_si` *SI Epidemic Model*

Description

Two-compartment model with no recovery.

Usage

```
model_si(init, params, times, ...)
```

Arguments

`init`	Named numeric vector, e.g. `c(S = 990, I = 10)`.
`params`	Named numeric vector, e.g. `c(beta = 0.002)`. **beta** Transmission rate (contacts per individual per time). **N** Total population size. Optional — defaults to `sum(init)`. **b** Birth rate (per capita per time). Optional. **d** Death rate (per capita per time). Optional.
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

Closed population (default):

$$\frac{dS}{dt} = -\beta \frac{SI}{N}$$

$$\frac{dI}{dt} = \beta \frac{SI}{N}$$

Open population (when b / d are provided):

$$\frac{dS}{dt} = bN - \beta \frac{SI}{N} - dS$$

$$\frac{dI}{dt} = \beta \frac{SI}{N} - dI$$

In the closed case population size $N = S + I$ is conserved.

Value

A `data.frame` with columns `time`, `S`, `I`.

Examples

```
# Closed population
model_si(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002),
  times  = seq(0, 30, by = 0.1)
)

# Open population with births and deaths
model_si(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, b = 0.01, d = 0.01),
  times  = seq(0, 200, by = 1)
)
```

`model_sir` *SIR Epidemic Model*

Description

Three-compartment model with permanent immunity after recovery.

Usage

```
model_sir(init, params, times, ...)
```

Arguments

`init`	Named numeric vector. `R` defaults to 0 if omitted, e.g. `c(S = 990, I = 10)`.
`params`	Named numeric vector, e.g. `c(beta = 0.002, gamma = 0.1)`.
	beta Transmission rate.
	gamma Recovery rate (1/gamma = mean infectious period).
	N Total population size. Optional — defaults to `sum(init)`.
	b Birth rate (per capita per time). Optional.
	d Death rate (per capita per time). Optional.
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

Closed population (default):

$$\frac{dS}{dt} = -\beta \frac{SI}{N}$$
$$\frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I$$
$$\frac{dR}{dt} = \gamma I$$

Open population (when b / d are provided):

$$\frac{dS}{dt} = bN - \beta \frac{SI}{N} - dS$$
$$\frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I - dI$$
$$\frac{dR}{dt} = \gamma I - dR$$

In the closed case population size $N = S + I + R$ is conserved. The basic reproduction number is $R_0 = \beta/\gamma$.

Value

A `data.frame` with columns `time`, `S`, `I`, `R`.

Examples

```
# Closed population
model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)

# Open population with births and deaths
model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1, b = 0.01, d = 0.01),
  times  = seq(0, 200, by = 1)
)
```

`model_sis` *SIS Epidemic Model*

Description

Two-compartment model with recovery but no lasting immunity.

Usage

```
model_sis(init, params, times, ...)
```

Arguments

`init`	Named numeric vector, e.g. `c(S = 990, I = 10)`.
`params`	Named numeric vector, e.g. `c(beta = 0.002, gamma = 0.1)`. **beta** Transmission rate. **gamma** Recovery rate (1/gamma = mean infectious period). **N** Total population size. Optional — defaults to `sum(init)`. **b** Birth rate (per capita per time). Optional. **d** Death rate (per capita per time). Optional.
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

Closed population (default):

$$\frac{dS}{dt} = -\beta \frac{SI}{N} + \gamma I$$

$$\frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I$$

Open population (when b / d are provided):

$$\frac{dS}{dt} = bN - \beta \frac{SI}{N} + \gamma I - dS$$

$$\frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I - dI$$

In the closed case population size $N = S + I$ is conserved.

Value

A `data.frame` with columns `time`, `S`, `I`.

Examples

```
# Closed population
model_sis(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)

# Open population with births and deaths
model_sis(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.3, gamma = 0.1, b = 0.01, d = 0.01),
  times  = seq(0, 200, by = 1)
)
```

`ode_solver` *General ODE Solver*

Description

Solves a system of ordinary differential equations (ODEs) using string-formula equations. Equation strings are parsed once at call time and reused across all integration steps, so the solver is substantially faster than approaches that call `parse()` inside the derivative function.

Usage

```
ode_solver(init, times, equations, params = NULL, method = "lsoda", ...)
```

Arguments

`init`	A named numeric vector of initial values for all state variables.
`times`	A numeric vector of output times.
`equations`	A named character vector; names are variable names, values are derivative expressions (e.g. `c(S = "-beta*S*I")`).
`params`	A named numeric vector or list of parameters referenced inside `equations`.
`method`	Integration method passed to `deSolve::ode`. Default `"lsoda"`.
`...`	Additional arguments forwarded to `deSolve::ode`.

Details

All equations are parsed with `parse()` exactly once before the integrator starts. A single pre-allocated environment is reused on every derivative evaluation, avoiding repeated memory allocation and garbage collection.

Value

A `data.frame` with a `time` column followed by one column per state variable.

Examples

```
# Built-in wrapper (one-liner)
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
head(sir)

# Custom SEIR model with demography
seir_demo = ode_solver(
  init  = c(S = 1000, E = 1, I = 0, R = 0),
  times = seq(0, 200, by = 1),
  equations = c(
    S = "mu*N - beta*S*I - mu*S",
    E = "beta*S*I - alpha*E - mu*E",
    I = "alpha*E - gamma*I - mu*I",
    R = "gamma*I - mu*R"
  ),
  params = c(beta = 0.3, alpha = 0.2, gamma = 0.1, mu = 0.01, N = 1000)
)
tail(seir_demo)
```

`pca_weight` *PCA-Based Weighting Method*

Description

Computes indicator weights using Principal Component Analysis (PCA). The method extracts principal components and uses their variance contribution to derive objective weights for indicators. Optionally handles positive/negative directions of indicators, and supports pre-standardized data.

Usage

```
pca_weight(X, index = NULL, nfs = NULL, varimax = TRUE, method = "abs")
```

Arguments

`X`	A numeric data frame or matrix where rows represent samples and columns represent indicators.
`index`	A character vector indicating the direction of each indicator. Use `"+"` for positive indicators (higher is better), `"-"` for negative indicators (lower is better), and `NA` for already standardized indicators (no standardization will be applied). `If `index = NULL` (default), all indicators are treated as `NA`, meaning no standardization is performed.`
`nfs`	Number of principal components to use; by default, all are used.
`varimax`	Whether to perform Varimax rotation, default is TRUE.
`method`	Weighting Method. "abs" uses absolute loading values `|a_{ji}|` (default), "squared" uses `a_{ji}^2`.

Value

A list containing:

w	Numeric vector of normalized weights for each indicator.
s	Numeric vector of scores for each sample.
lambda	Eigenvalues of principal components (explained variance).
varP	Proportion of variance explained by selected PCs.

Examples

```
# Example: Using PCA to compute indicator weights
ind = c("+","+","-","-")
pca_weight(iris[1:10, 1:4], ind, nfs = 2)
```

---

plot_compartments *Plot compartment trajectories*

Description

Draws one line per compartment over time. Use `compartments` to select a subset of the state variables; the default is to plot all columns except `time`.

Usage

```
plot_compartments(df, compartments = NULL)
```

Arguments

df	A data frame returned by `ode_solver()` or any built-in model function.
compartments	Character vector of compartment names to plot. When `NULL` (the default) every column except `time` is plotted.

Value

A ggplot object.

Examples

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
plot_compartments(sir)
plot_compartments(sir, compartments = c("S", "I"))
```

plot_incidence *Plot daily new infections (dI)*

Description

Computes the daily change in the infectious compartment ΔI directly via `diff()` and plots it as a line chart. The peak is highlighted with a point marker.

Usage

```
plot_incidence(df)
```

Arguments

`df`	A data frame returned by `ode_solver()` or any built-in model function. Must contain columns `time` and `I`.

Value

A ggplot object.

Examples

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
plot_incidence(sir)
```

plot_phase_si *Phase plot S vs I*

Description

Draws a trajectory in the (S, I) plane. Useful for visualising the epidemic orbit and threshold behaviour.

Usage

```
plot_phase_si(df)
```

Arguments

`df`	A data frame with columns `time`, `S`, and `I`.

Value

A ggplot object.

Examples

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
plot_phase_si(sir)
```

---

plot_reg_predict *Prediction Plot*

Description

Returns a fitted-vs-observed line plot, with optional confidence bands.

Usage

```
plot_reg_predict(model_result, type = c("fitted", "predicted"), show_ci = TRUE)
```

Arguments

model_result	A fitted result from any reg_*() function.
type	Character. Currently supports "fitted" only.
show_ci	Logical. Whether to overlay confidence intervals. Default TRUE.

Value

A ggplot object.

---

plot_reg_residuals *Residual Diagnostics*

Description

Returns a multi-panel diagnostic plot from a fitted regression model.

Usage

```
plot_reg_residuals(model_result)
```

Arguments

model_result	A list returned by any reg_*() function.

Value

A ggplot-based composite.

plot_Rt_estimate *Plot effective reproduction number R_t*

Description

Estimates the time-varying effective reproduction number R_t directly from compartment data. Two methods are available:

`"mechanistic"` Uses $R_t = \beta S(t)/(\gamma N)$. This is the standard formula for a frequency-dependent SIR-type model.

`"normalized"` Uses $R_t = R_0\, S(t)/N$ with $R_0 = \beta/\gamma$. Suitable when different normalisations are desired.

Usage

```
plot_Rt_estimate(df, params, N = NULL, method = c("mechanistic", "normalized"))
```

Arguments

`df`	A data frame with columns `time` and `S`.
`params`	A named numeric vector or list containing at least `beta` and `gamma`.
`N`	Total population size. If `NULL` (the default), it is estimated from the sum of all state columns at the first time step.
`method`	Estimation method: `"mechanistic"` (default) or `"normalized"`.

Value

A ggplot object with a dashed horizontal line at $R_t = 1$.

Examples

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
plot_Rt_estimate(sir, params = c(beta = 0.002, gamma = 0.1))
```

plot_ts *Plot a Time Series*

Description

Plot a Time Series

Usage

```
plot_ts(x, title = "Time Series", x_lab = "Time", y_lab = "Value")
```

Arguments

x	A ts_df.
title	Plot title.
x_lab, y_lab	Axis labels.

Value

A ggplot object.

Examples

```
plot_ts(as_ts_df(AirPassengers))
```

---

plot_ts_acf *Autocorrelation Plot*

Description

Autocorrelation Plot

Usage

```
plot_ts_acf(x, max_lag = 40L, title = "ACF")
```

Arguments

x	A complete ts_df with no missing values.
max_lag	Maximum lag.
title	Plot title.

Value

A ggplot object.

Examples

```
plot_ts_acf(as_ts_df(log(AirPassengers)))
```

plot_ts_forecast *Plot Forecasts*

Description

Plot Forecasts

Usage

```
plot_ts_forecast(
  x,
  forecast_tbl,
  level = c(80, 95),
  by = NULL,
  title = "Forecast"
)
```

Arguments

x	Original ts_df.
forecast_tbl	Result from ts_forecast().
level	Confidence levels to draw when present.
by	Step size for future time values.
title	Plot title.

Value

A ggplot object.

Examples

```
fit = ts_ets(as_ts_df(log(AirPassengers)))
fc = ts_forecast(fit, h = 3)
plot_ts_forecast(as_ts_df(log(AirPassengers)), fc, by = 1)
```

plot_ts_pacf *Partial Autocorrelation Plot*

Description

Partial Autocorrelation Plot

Usage

```
plot_ts_pacf(x, max_lag = 40L, title = "PACF")
```

Arguments

x	A complete ts_df with no missing values.
max_lag	Maximum lag.
title	Plot title.

Value

A ggplot object.

Examples

```
plot_ts_pacf(as_ts_df(log(AirPassengers)))
```

---

plot_ts_residuals *Residual Diagnostic Plot*

Description

Residual Diagnostic Plot

Usage

```
plot_ts_residuals(model_result, max_lag = 30L, title = "Residual Diagnostics")
```

Arguments

model_result	Result from ts_ets() or ts_sarima().
max_lag	Maximum lag.
title	Plot title.

Value

A patchwork plot.

Examples

```
fit = ts_sarima(as_ts_df(log(AirPassengers)), stepwise = TRUE, approximation = TRUE)
plot_ts_residuals(fit)
```

plot_ts_stl *STL Decomposition Plot*

Description

STL Decomposition Plot

Usage

```
plot_ts_stl(stl_result, title = "STL Decomposition")
```

Arguments

stl_result	Result from ts_stl().
title	Plot title.

Value

A patchwork plot.

Examples

```
plot_ts_stl(ts_stl(as_ts_df(AirPassengers)))
```

poly_fit *Polynomial Fit*

Description

Fits a polynomial regression model via lm().

Usage

```
poly_fit(x, y, degree = 1)
```

Arguments

x	Numeric vector of observed predictor values.
y	Numeric vector of observed response values.
degree	Integer, degree of the polynomial. Default is 1 (linear).

Value

A named list with elements: model (lm object), coefficient (tibble with estimates, SE, t-value, p-value, 95% CI), model_info (tibble with r.squared, adj.r.squared, AIC, BIC, sigma), residuals (tibble of .observed, .fitted, .residual), formula (character), type (character), and input (tibble of x, y).

Examples

```
x = 1:20
y = 3 + 2*x + rnorm(20, sd = 2)
poly_fit(x, y, degree = 1)
poly_fit(x, y, degree = 2)
```

`preprocess` *Preprocessing Functions for Data Normalization and Standardization*

Description

A collection of functions to preprocess numeric data, including standardization, L2 norm normalization, Min-Max scaling, centered-type normalization, interval-type normalization, extreme-value-based normalization, initial-value-based normalization, mean-based normalization, and negative-to-positive transformation. These functions transform a numeric vector to a standardized or normalized scale, suitable for various indicator types (positive, negative, centered, interval-based, or extreme-based).

Usage

```
standardize(x, center = TRUE, scale = TRUE)

normalize(x)

rescale(x, type = "+", a = 0, b = 1)

rescale_middle(x, m)

rescale_interval(x, a, b)

rescale_extreme(x, type = "+")

rescale_initial(x, type = "+")

rescale_mean(x)

to_positive(x, type = "minmax")
```

Arguments

`x`	Numeric vector to be preprocessed.
`center`	Logical or numeric scalar, passed to `base::scale` for centering (for `standardize`). Default is `TRUE`.
`scale`	Logical or numeric scalar, passed to `base::scale` for scaling (for `standardize`). Default is `TRUE`.
`type`	Character scalar specifying the transformation direction or method:
	`"+"` Positive direction (larger values are better, for `rescale`, `rescale_extreme` and `rescale_initial`).

	"-" Negative direction (smaller values are better, for `rescale` `rescale_extreme` and `rescale_initial`).
	"minmax" Min-max transformation (for `to_positive`).
	"reciprocal" Reciprocal transformation (for `to_positive`).
`a`	Numeric scalar, lower bound of the output range or optimal interval (for `rescale` and `rescale_interval`).
`b`	Numeric scalar, upper bound of the output range or optimal interval (for `rescale` and `rescale_interval`).
`m`	Numeric scalar, optimal value for centered-type normalization (for `rescale_middle`).

Details

These functions support various preprocessing needs in data analysis:

- `standardize`: Applies Z-score standardization (mean = 0, sd = 1), ideal for equalizing variances or normally distributed data.
- `normalize`: Scales the vector to unit length by dividing by its L2 (Euclidean) norm, useful for machine learning or similarity calculations.
- `rescale`: Performs Min-Max scaling to a specified range (default `[0, 1]`), supporting positive or negative indicators.
- `rescale_middle`: Normalizes centered-type indicators, where values closer to an optimal value `m` are better, mapping to `[0, 1]`.
- `rescale_interval`: Normalizes interval-type indicators, where values within `[a, b]` are optimal, mapping to `[0, 1]`.
- `rescale_extreme`: Normalizes using extreme values: `min(x)/x` for positive indicators or `x/max(x)` for negative indicators, often used in grey relational analysis.
- `rescale_initial`: Normalizes by dividing by the first value (`x/x[1]` or `x[1]/x`), commonly used in grey relational analysis.
- `rescale_mean`: Normalizes by dividing by the mean (`x/mean(x)`), commonly used in grey relational analysis.
- `to_positive`: Converts negative indicators to positive using either min-max (`max(x) - x`) or reciprocal (`1/x`) transformation.

Missing values (`NA`) are preserved in the output. For `rescale_initial` and `rescale_mean`, the initial value or mean must be non-zero, respectively.

Value

A numeric vector of the same length as `x`, transformed as follows:

- `standardize`: Standardized values (mean = 0, sd = 1).
- `normalize`: L2 norm normalized values (Euclidean norm, unit length).
- `rescale`: Min-Max scaled values in `[a, b]` (default `[0, 1]`).
- `rescale_middle`: Centered-type normalized values in `[0, 1]`, where 1 indicates `x = m`.
- `rescale_interval`: Interval-type normalized values in `[0, 1]`, where 1 indicates `x` in `[a, b]`.
- `rescale_extreme`: Extreme-based normalized values using `min(x)/x` (positive) or `x/max(x)` (negative).
- `rescale_initial`: Initial-based normalized values using `x/x[1]` or `x[1]/x`.
- `rescale_mean`: Mean-based normalized values using `x/mean(x)`.
- `to_positive`: Transformed values converting negative indicators to positive using min-max or reciprocal transformation.

Examples

```
# Standardization
x = c(4, 1, NA, 5, 8)
standardize(x)

# L2 norm normalization
normalize(x)

# Min-Max normalization (positive direction)
rescale(x)                # Scale to \code{[0, 1]}
rescale(x, type = "-", a = 0.002, b = 0.996)  # Reverse scaling

# Negative-to-positive transformation
to_positive(x)                       # Min-max transformation
to_positive(x, type = "reciprocal")  # Reciprocal transformation

# Centered-type normalization
PH = 6:9
rescale_middle(PH, 7)

# Interval-type normalization
Temp = c(35.2, 35.8, 36.6, 37.1, 37.8, 38.4)
rescale_interval(Temp, 36, 37)

# Extreme-based normalization
rescale_extreme(x)         # min(x)/x
rescale_extreme(x, "-")    # x/max(x)

# Initial-based normalization
rescale_initial(x)

# Mean-based normalization
rescale_mean(x)
```

`rank_sum_ratio` *Rank Sum Ratio (RSR) Evaluation*

Description

Performs Rank Sum Ratio (RSR) evaluation on a dataset of positive indicators, computing ranks, weighted RSR values, and a linear regression model to fit RSR against probit-transformed ranks. Supports integer or non-integer ranking methods.

Usage

```
rank_sum_ratio(data, w = NULL, method = "int")
```

Arguments

`data`	Data frame with positive indicator data; first column is an ID column for identifying evaluation objects.
`w`	Numeric vector, weights for indicators (default = equal weights).

method	Character scalar, ranking method: "int" for integer ranks or "non-int" for scaled ranks in [1, n] (default = "int").

Details

The rank_sum_ratio function implements the RSR method for evaluating objects based on positive indicators. It ranks the indicators (using integer or non-integer methods), computes weighted RSR values, adjusts ranks with probit transformation, and fits a linear regression model to relate RSR to probit values. The function assumes the first column of data is an ID column, and weights (w) can be provided or set to equal weights by default.

Value

A list containing:

- resultTable: Data frame with RSR values, ranks, cumulative frequencies, probit values, and fitted RSR values.
- reg: Linear model object fitting RSR against probit values.
- rankTable: Data frame with ranked indicator values.

Examples

```
# Example data
data = data.frame(ID = c("A", "B", "C"), X1 = c(10, 20, 15), X2 = c(5, 10, 8))
w = c(0.4, 0.6)
rank_sum_ratio(data, w, method = "int")
```

---

read_nbs *Read and Combine National Bureau of Statistics XLS Files*

Description

This function reads multiple XLS files downloaded from the National Bureau of Statistics of China website (https://www.stats.gov.cn) without requiring renaming. It extracts the variable name from the "indicator: XXX" text in cell A1 of each file, skips the first 3 rows of headers, reads the first 31 rows of data, and combines the data into a single data frame by stacking vertically based on the indicator names. It also simplifies region names by removing suffixes such as "city", "province", "autonomous region", "clan", or "Uyghur".

Usage

```
read_nbs(paths)
```

Arguments

paths	A character vector containing the file paths to the XLS files.

Value

A tibble containing the combined data with an additional column indicator representing the original indicator names and a region column with simplified region names.

Examples

```
## Not run:
paths = c("file1.xls", "file2.xls")
data = read_nbs(paths)

## End(Not run)
```

`regional_economics` *Regional Economics Functions*

Description

A collection of functions for calculating regional economics indices, including Location Quotient (LQ), Herfindahl-Hirschman Index (HHI), and Ellison-Glaeser Index (EG). These functions are designed for analyzing regional and industrial data to assess spatial concentration and market structure.

Usage

```
LQ(data, region, total, .cols, .by = NULL)

HHI(x, scaled = FALSE)

EG(data, region, industry, y)
```

Arguments

`data`	A data frame containing the necessary data for calculations. For `LQ`, the first row is assumed to be the national total, with the first two columns specifying the region and total, optionally including a grouping column. For `EG`, it contains region, industry, and indicator columns (e.g., employment or output).
`region`	The name of the column in `data` specifying the region (used in `LQ` and `EG`).
`total`	The name of the column in `data` specifying the total (used in `LQ`).
`.cols`	The columns in `data` for which to calculate the location quotient (used in `LQ`).
`.by`	Optional grouping column for `LQ`, defaults to `NULL` (no grouping).
`x`	A numeric vector for calculating the HHI (used in `HHI`).
`scaled`	Logical; if `TRUE`, the HHI is scaled to account for the number of firms. Defaults to `FALSE`.
`industry`	The name of the column in `data` specifying the industry (used in `EG`).
`y`	The name of the column in `data` specifying the indicator (e.g., employment or output, used in `EG`).

Details

LQ Calculates the Location Quotient for multiple columns with optional grouping. The LQ measures the relative concentration of an industry in a region compared to a national benchmark. The function assumes the first row of `data` contains national totals, with the first two columns specifying the region and total, and the remaining columns used for LQ calculation.

HHI Calculates the Herfindahl-Hirschman Index, a measure of market concentration based on the squared sum of market shares. If `scaled = TRUE`, the HHI is normalized to account for the number of firms.

EG Calculates the Ellison-Glaeser Index, which measures the geographic concentration of an industry while controlling for firm size distribution and random distribution effects. It requires data on regions, industries, and an indicator (e.g., employment or output).

Value

LQ A data frame with the region column and calculated location quotients for the specified columns, optionally grouped by `.by`.

HHI A numeric value representing the HHI, either scaled or unscaled.

EG A tibble with two columns: the industry name and the corresponding EG index value.

Examples

```
# Example data
data = data.frame(
  region = c("National", "Region_A", "Region_B"),
  total = c(10000, 4000, 6000),
  industry1 = c(2000, 1000, 1000),
  industry2 = c(6000, 2000, 4000)
)

# Calculate Location Quotient
LQ(data, region, total, starts_with("industry"))

# Calculate HHI
x = c(50, 30, 20)
# Calculate the raw HHI
HHI(x)
# Calculate the standard (scaled) HHI
HHI(x, scaled = TRUE)

# Example data for EG
eg_data = data.frame(
 region = c("R1", "R1", "R1", "R1", "R2", "R2", "R3", "R3", "R1", "R2", "R2", "R3"),
 industry = c("A", "A", "A", "A", "A", "A", "A", "A", "B", "B", "B", "B"),
 employment = c(250, 200, 150, 100, 20, 15, 10, 5, 50, 200, 150, 50)
)
EG(eg_data, region, industry, y = employment)
```

---

reg_diagnostics *Model Diagnostics*

Description

Runs diagnostic tests on a fitted regression model.

Usage

```
reg_diagnostics(model_result)
```

Arguments

model_result	A list returned by any reg_*() function.

Value

A named list with diagnostic tibbles. For linear models: VIF, Shapiro-Wilk normality, Breusch-Pagan, Durbin-Watson, residual stats. For logistic/poisson/negative binomial: Hosmer-Lemeshow / dispersion / deviance stats.

---

reg_lm *Multivariable Linear Regression*

Description

Fits an OLS model via lm(), optionally performing stepwise selection using Akaike's information criterion.

Usage

```
reg_lm(
  formula,
  data,
  step = FALSE,
  direction = c("both", "forward", "backward"),
  ...
)
```

Arguments

formula	A formula, e.g. y ~ x1 + x2.
data	A data frame containing the variables in formula.
step	Logical. Whether to perform stepwise selection (TRUE/FALSE). Default is FALSE.
direction	Character: "both" (default), "forward", or "backward".
...	Additional arguments passed to lm() and step().

Details

When step = TRUE, the function first fits a null model (response only), then uses step() to search for the best combination of terms by AIC.

Value

A named list:

model Fitted lm object.

coefficient Tibble of coefficients, standard errors, t-values, p-values, 95% CIs.

model_info Tibble of fit statistics (r.squared, adj.r.squared, AIC, BIC).

residuals Tibble of observed, fitted, residuals.

diagnostics Tibble of key fit metrics (Breusch-Pagan, Durbin-Watson).

formula Original formula.

input The original data frame.

Examples

```
set.seed(42)
x1 = rnorm(100); x2 = rnorm(100)
y = 1 + 2*x1 - 3*x2 + rnorm(100, sd = 2)
reg_lm(y ~ x1 + x2, data = data.frame(y, x1, x2))
```

---

reg_logistic *Logistic Regression*

Description

Fits a binary logistic regression model via `glm(family = binomial())`, with optional stepwise selection.

Usage

```
reg_logistic(
  formula,
  data,
  step = FALSE,
  direction = c("both", "forward", "backward"),
  ...
)
```

Arguments

`formula`	A formula, e.g. `y ~ x1 + x2`.
`data`	A data frame containing the variables in `formula`.
`step`	Logical. Whether to perform stepwise selection (`TRUE`/`FALSE`). Default is `FALSE`.
`direction`	Character: `"both"` (default), `"forward"`, or `"backward"`.
`...`	Additional arguments passed to `lm()` and `step()`.

Details

Same stepwise logic as `reg_lm`, but uses GLM with the binomial family. A Hosmer-Lemeshow goodness-of-fit test is included in diagnostics.

Value

A named list analogous to `reg_lm()`, plus:

odds_ratio Tibble with odds ratios and 95% CIs.

residuals DataFrame of observed, fitted probability, residual.

Examples

```
set.seed(42)
x1 = rnorm(100)
logit = 0.5 + x1
prob = 1 / (1 + exp(-logit))
y = rbinom(100, 1, prob)
reg_logistic(y ~ x1, data = data.frame(y, x1))
```

---

reg_negbin *Negative Binomial Regression*

Description

Fits a negative binomial regression model (for over-dispersed counts) via MASS::glm.nb().

Usage

```
reg_negbin(formula, data, step = FALSE, ...)
```

Arguments

formula	A formula, e.g. count ~ x1 + x2.
data	A data frame.
step	Logical. Always FALSE for negative binomial.
...	Additional arguments passed to MASS::glm.nb().

Details

Note: The glm.nb family does not support stepwise selection. Set step = FALSE.

For comparison with Poisson, inspect the dispersion ratio. NB is appropriate when dispersion » 1.

Value

A named list analogous to reg_poisson(), plus:

theta Estimated dispersion parameter for the negative binomial distribution.

residuals DataFrame of observed, fitted, Pearson residual.

Examples

```
set.seed(42)
x1 = rnorm(100)
mu = exp(1 + 0.3*x1)
y = rnbinom(100, size = 1, mu = mu)
reg_negbin(y ~ x1, data = data.frame(y, x1))
```

`reg_poisson` *Poisson Regression*

Description

Fits a count-response regression model via `glm(family = poisson())`, with optional stepwise selection.

Usage

```
reg_poisson(
  formula,
  data,
  step = FALSE,
  direction = c("both", "forward", "backward"),
  ...
)
```

Arguments

`formula`	A formula, e.g. `y ~ x1 + x2`.
`data`	A data frame containing the variables in `formula`.
`step`	Logical. Whether to perform stepwise selection (`TRUE`/`FALSE`). Default is `FALSE`.
`direction`	Character: `"both"` (default), `"forward"`, or `"backward"`.
`...`	Additional arguments passed to `lm()` and `step()`.

Details

Performs an overdispersion check: dispersion > 1.5 suggests NB may be preferable.

Value

A named list analogous to `reg_lm()`, plus:

dispersion Dispersion ratio (should be near 1).

residuals Data frame with observed, fitted, Pearson residual.

Examples

```
set.seed(42)
x1 = rnorm(100)
mu = exp(1 + 0.3*x1)
y = rpois(100, mu)
reg_poisson(y ~ x1, data = data.frame(y, x1))
```

reg_predict *Predictions*

Description

Generates point predictions and confidence/ prediction intervals for new data.

Usage

```
reg_predict(model_result, n_new = 10, level = 0.95)
```

Arguments

model_result	A fitted result from any reg_*() function.
n_new	Number of new observations to predict. Random values drawn from the fitted model's residuals are used for predictor data.
level	Confidence/interval level. Defaults to 0.95.

Value

A named list with:

predictions Tibble with predicted values.

confidence_interval Tibble with lower and upper bounds.

system_evaluation *System Evaluation Functions for Coupling and Obstacle Analysis*

Description

These functions provide tools for system-level evaluation in multi-indicator systems:

- coupling_degree(): Computes coupling degree, coordination index, and coupling coordination degree for subsystems.
- obstacle_degree(): Computes obstacle degrees for secondary indicators to identify key constraints in the system, enabling batch processing with tidyverse for grouping and summarization.

Usage

```
coupling_degree(data, w = NULL, id_cols = NULL, type = "standard")

obstacle_degree(data, w = NULL, id_cols = NULL, scaled = FALSE)
```

Arguments

data	A data frame with normalized scores (usually in [0, 1]) as columns.
w	Optional vector of weights for indicators or subsystems; defaults to equal weights if NULL.
id_cols	Optional character vector of column names in data to preserve as identifiers (not used in calculations).
type	Either "standard" for the standard coupling formula (results concentrated near 1) or "adjusted" for the revised formula (results more uniformly distributed in [0, 1]), as proposed by Wang Shujia, Kong Wei, et al. in "Misconceptions and Corrections of Domestic Coupling Coordination Degree Models, Journal of Natural Resources, 2021, 36(3): 793–810 (In Chinese)"
scaled	Logical. Whether to perform row normalization on obstacle degrees (default FALSE).

Value

A tibble depending on the function:

coupling_degree A tibble with columns:

- ID: Identifier columns specified by id_cols (if provided).
- coupling: Coupling Degree (range 0-1).
- coord: Coordination Index (range 0-1).
- coupling_coord: Coupling Coordination Degree (range 0-1).

obstacle_degree A tibble with:

- ID: Identifier columns specified by id_cols (if provided).
- Columns for secondary indicator obstacle degrees (O_{ij} = (1 - X_{ij}) * w_{ij}).

Suitable for grouping and summarizing (e.g., with tidyverse) to compute primary indicator obstacle degrees (\(U_i \)).

Examples

```
# Sample normalized subsystem scores
df = data.frame(
  ID = LETTERS[1:6],
  s1 = c(0.0162, 0.1782, 0.5490, 0.6730, 0.0207, 0.9875),
  s2 = c(0.2720, 0.6824, 0.0593, 0.4812, 0.8891, 0.5573),
  s3 = c(0.2655, 0.3721, 0.5729, 0.9082, 0.2017, 0.8984)
)
# Coupling Degree Analysis
coupling_degree(df, id_cols = "ID")        # Equal weights
coupling_degree(df, c(0.4, 0.3, 0.3), id_cols = "ID",
                type = "adjusted")         # "adjusted" coupling degree
# Obstacle Degree Analysis
obstacle_degree(df, id_cols = "ID")        # Equal weights
obstacle_degree(df, c(0.4, 0.3, 0.3), id_cols = "ID")
```

topsis *TOPSIS Method for Multi-Criteria Decision Making*

Description

Implements the Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS) to rank alternatives based on multiple criteria. The function normalizes the decision matrix using L2 norm, applies weights, and computes relative closeness to the ideal solution.

Usage

```
topsis(X, w = NULL, index = NULL)
```

Arguments

X	A numeric matrix or data frame where rows represent alternatives and columns represent criteria.
w	A numeric vector of weights for each criterion. Must be non-negative and sum to 1. If not provided, equal weights are used.
index	A character vector indicating the direction of each indicator: Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better). If index = NULL (default), all indicators are treated as "+".

Details

The TOPSIS method ranks alternatives by:

1. Normalizing the decision matrix using L2 norm normalization.
2. Applying weights to form a weighted normalized matrix.
3. Identifying positive and negative ideal solutions based on indicator directions.
4. Computing Euclidean distances to ideal solutions.
5. Calculating relative closeness as S0 / (S0 + Sstar), where S0 is the distance to the negative ideal and Sstar is the distance to the positive ideal.

This implementation supports both positive and negative indicators via the index parameter.

Value

A named numeric vector of relative closeness scores (in [0, 1]) for each alternative. Higher values indicate better alternatives. Names are taken from rownames(X) or default to "Sample1", "Sample2", etc.

Examples

```
A = data.frame(
  X1 = c(2, 5, 3),  # "+"
  X2 = c(8, 1, 6)   # "-"
)
w = c(0.6, 0.4)
idx = c("+","-")
topsis(A, w, idx)
```

`ts_arimax` *ARIMAX Model Fitting*

Description

Fits an ARIMA model with exogenous regressors (ARIMAX).

Usage

```
ts_arimax(
  x,
  xreg,
  order = NULL,
  seasonal = NULL,
  stepwise = TRUE,
  approximation = TRUE,
  ...
)
```

Arguments

`x`	A complete `ts_df` with no missing values.
`xreg`	A matrix or data.frame of exogenous regressors with `nrow(xreg) == nrow(x)`.
`order`	Optional non-seasonal ARIMA order. If `NULL` (default), automatic order selection via `forecast::auto.arima()`.
`seasonal`	Optional seasonal specification.
`stepwise, approximation`	Passed to `forecast::auto.arima()`.
`...`	Additional arguments passed to the forecast fitting function.

Value

A list containing tidy model summaries, the raw model, input, and the exogenous regressors used.

Examples

```
x = as_ts_df(log(AirPassengers))
xreg = data.frame(trend = seq_len(nrow(x)))
ts_arimax(x, xreg = xreg)
```

ts_back_transform *Back-Transform Forecasts*

Description

Converts forecasts produced on a transformed scale back to the original scale.

Usage

```
ts_back_transform(forecast_tbl, params)
```

Arguments

forecast_tbl	Forecast tibble from ts_forecast().
params	The params element from ts_transform().

Value

The forecast tibble with numeric forecast columns restored.

Examples

```
x = as_ts_df(AirPassengers)
tr = ts_transform(x, method = "log")
fc = tibble::tibble(step = 1:2, forecast = log(c(500, 600)))
ts_back_transform(fc, tr$params)
```

ts_df *Lightweight Time-Series Data Frame*

Description

Creates a simple single-series time-series data frame with fixed columns time and value, plus frequency and start attributes.

Usage

```
ts_df(data, time = "time", value = "value", frequency = 1, start = NULL)
```

Arguments

data	A data frame.
time	Name of the time/index column.
value	Name of the numeric value column.
frequency	Positive numeric scalar. Defaults to 1.
start	Optional start metadata. If NULL, uses the first sorted time.

Value

A ts_df object, which is also a data frame.

Examples

```
df = data.frame(time = c(2, 1, 3), value = c(12, 10, 15))
ts_df(df)
```

---

ts_ets *ETS Model Fitting*

Description

ETS Model Fitting

Usage

```
ts_ets(x, model = "ZZZ", ...)
```

Arguments

x	A complete ts_df with no missing values.
model	ETS model string passed to forecast::ets().
...	Additional arguments passed to forecast::ets().

Value

A list containing tidy model summaries, the raw model, and input.

Examples

```
ts_ets(as_ts_df(log(AirPassengers)))
```

---

ts_forecast *Generate Forecasts*

Description

Generate Forecasts

Usage

```
ts_forecast(model_result, h = 12, level = c(80, 95), newxreg = NULL)
```

Arguments

model_result	Result from ts_ets(), ts_sarima(), or ts_arimax().
h	Forecast horizon.
level	Confidence levels.
newxreg	Future exogenous regressors for ARIMAX models. A matrix or data.frame with nrow(newxreg) == h and the same number of columns as the original xreg.

Value

A tibble with point forecasts and intervals.

Examples

```
fit = ts_ets(as_ts_df(log(AirPassengers)))
ts_forecast(fit, h = 3)

x = as_ts_df(log(AirPassengers))
xreg = data.frame(trend = seq_len(nrow(x)))
fit_arimax = ts_arimax(x, xreg = xreg, order = c(1, 1, 1))
newxreg = data.frame(trend = nrow(x) + 1:3)
ts_forecast(fit_arimax, h = 3, newxreg = newxreg)
```

---

ts_sarima *SARIMA Model Fitting*

Description

SARIMA Model Fitting

Usage

```
ts_sarima(
  x,
  order = NULL,
  seasonal = NULL,
  stepwise = TRUE,
  approximation = TRUE,
  ...
)
```

Arguments

x	A complete ts_df with no missing values.
order	Optional non-seasonal ARIMA order.
seasonal	Optional seasonal specification.
stepwise, approximation	Passed to forecast::auto.arima().
...	Additional arguments passed to the forecast fitting function.

Value

A list containing tidy model summaries, the raw model, and input.

Examples

```
ts_sarima(as_ts_df(log(AirPassengers)), stepwise = TRUE, approximation = TRUE)
```

ts_stl *STL Decomposition for a* ts_df

Description

Decomposes a complete seasonal ts_df with stats::stl().

Usage

```
ts_stl(x, s_window = "periodic", robust = TRUE, ...)
```

Arguments

x	A complete seasonal ts_df with no missing values.
s_window	Seasonal smoothing window.
robust	Use robust STL fitting.
...	Additional arguments passed to stats::stl().

Value

A list with components, strength, and model.

Examples

```
ts_stl(as_ts_df(AirPassengers))
```

ts_test *Stationarity Tests for a Time Series*

Description

Runs ADF and KPSS tests on a complete ts_df.

Usage

```
ts_test(x, adf_lags = NULL)
```

Arguments

x	A complete ts_df with no missing values.
adf_lags	Optional lag for the ADF test.

Value

A tibble with test results.

Examples

```
ts_test(as_ts_df(log(AirPassengers)))
```

ts_transform *Transform a* ts_df

Description

Applies a variance-stabilising transform and optional differencing. Returns a transformed ts_df plus explicit parameters for back-transformation.

Usage

```
ts_transform(
  x,
  method = c("none", "log", "boxcox"),
  lambda = NULL,
  diff = 0L,
  seasonal_diff = 0L
)
```

Arguments

x	A complete ts_df with no missing values.
method	One of "none", "log", or "boxcox".
lambda	Optional Box-Cox lambda.
diff	Number of regular differences.
seasonal_diff	Number of seasonal differences.

Value

A list with transformed, params, and summary.

Examples

```
ts_transform(as_ts_df(AirPassengers), method = "log", diff = 1)
```

validate_ts_df *Validate a* ts_df

Description

Validate a ts_df

Usage

```
validate_ts_df(x, require_complete = TRUE, require_no_na = FALSE)
```

Arguments

x	A ts_df object.
require_complete	If TRUE, first and last values must not be NA.
require_no_na	If TRUE, no values may be NA.

Value

A sorted ts_df object.

Examples

```
x = ts_df(data.frame(time = 1:3, value = c(10, 12, 15)))
validate_ts_df(x)
```

---

water_quality *Water Quality Dataset*

Description

A dataset containing water quality evaluation metrics for 20 rivers, including dissolved oxygen (O2, positive indicator), pH value (PH, centered indicator), total bacteria count (germ, negative indicator), and plant nutrient content (nutrient, interval indicator with optimal range 10-20). This dataset is suitable for multi-criteria decision analysis, such as weight calculation and fuzzy comprehensive evaluation in the mathmodels package.

Usage

```
water_quality
```

Format

A data frame with 20 rows and 5 columns:

ID Numeric, unique identifier for each river (1 to 20).

O2 Numeric, dissolved oxygen content (mg/L), higher values are better (positive indicator).

PH Numeric, pH value, values closer to 7 are optimal (centered indicator).

germ Numeric, total bacteria count, lower values are better (negative indicator).

nutrient Numeric, plant nutrient content (mg/L), optimal range is 10-20 (interval indicator).

Details

Water Quality Dataset

Source

Simulated data for water quality evaluation, created for demonstration purposes.

Examples

```
# Load the dataset
data(water_quality)

# Preview the data
head(water_quality)
```

Index